

State-Space Advanced Design Problem

A full output-feedback design from start to finish.

Ogata, *Modern Control Engineering*, Ch. 10.

In this tutorial you will:

- design a state-feedback gain and a fast observer for one plant,
- assemble the observer-based compensator and check the separation principle, and
- confirm the compensator reproduces the intended closed-loop poles.

Step through with **Ctrl+Enter**, or render a report with **publish**.

Design problem: for the plant

$$G(s) = \frac{1}{s(s+1)(s+5)}$$

only the output y is measured. Design (a) a state-feedback gain K placing the closed-loop poles at $s = -4 \pm 4j$, -10 , (b) a full-order observer with poles at $-20, -21, -22$ (well faster than the controller poles), and (c) the resulting observer-based compensator, then verify the separation principle and simulate the regulator response to a nonzero initial condition.

Contents

- Plant in Controllable Canonical Form
- (a) State-Feedback Gain
- (b) Observer Gain
- (c) Observer-Based Compensator
- Separation Principle Verification
- Regulator Simulation from Nonzero Initial Condition
- Before vs. After: Uncontrolled Plant vs. Observer-Based Regulator
- Closed-Loop Transfer Function via the Compensator
- Try it yourself

Plant in Controllable Canonical Form

```
G = tf(1,conv([1 0],conv([1 1],[1 5])));
sys_ccf = ss(G);
A = sys_ccf.A; B = sys_ccf.B; C = sys_ccf.C; D = sys_ccf.D;
n = size(A,1);

fprintf('Open-loop poles: '); disp(eig(A))
fprintf('Controllability rank = %d, Observability rank = %d (n=%d)\n', ...
        rank(ctrb(A,B)), rank(observ(A,C)), n)
```

```
Open-loop poles:      0      -1      -5
```

```
Controllability rank = 3, Observability rank = 3 (n=3)
```

(a) State-Feedback Gain

```
controller_poles = [-4+4j,-4-4j,-10];
K = acker(A,B,controller_poles);
fprintf('\nController gain K = '); disp(K)
```

```
Controller gain K =      12.0000      53.5000      320.0000
```

(b) Observer Gain

```
observer_poles = [-20,-21,-22];
G_obs = acker(A',C',observer_poles)';
fprintf('Observer gain G = '); disp(G_obs)
```

```
Observer gain G =          3105
                  1950
                  57
```

(c) Observer-Based Compensator

$G_c(s) = K(sI - A + BK + GC)^{-1}G$, implementing $u = -K\hat{x}$ with the observer's dynamics eliminated as an explicit plant-visible state.

```
A_c = A - B*K - G_obs*C;
B_c = G_obs;
C_c = -K;
D_c = 0;
Gc = tf(ss(A_c,B_c,C_c,D_c));
fprintf('\nCompensator transfer function:\n')
Gc
```

Compensator transfer function:

Gc =

```
-1.598e05 s^2 - 1.321e06 s - 2.957e06
-----
s^3 + 75 s^2 + 2113 s + 2.736e04
```

Continuous-time transfer function.

Separation Principle Verification

```
A_aug = [A-B*K, B*K; zeros(n), A-G_obs*C];
eig_aug = eig(A_aug);
fprintf('\nCombined eigenvalues:\n'); disp(eig_aug')
fprintf('Expected (controller U observer poles):\n')
disp([controller_poles observer_poles])
```

Combined eigenvalues:

Columns 1 through 4

```
-10.0000 + 0.0000i -4.0000 - 4.0000i -4.0000 + 4.0000i -20.0000 + 0.0000i
```

Columns 5 through 6

```
-21.0000 + 0.0000i -22.0000 + 0.0000i
```

Expected (controller U observer poles):

Columns 1 through 4

```
-4.0000 + 4.0000i -4.0000 - 4.0000i -10.0000 + 0.0000i -20.0000 + 0.0000i
```

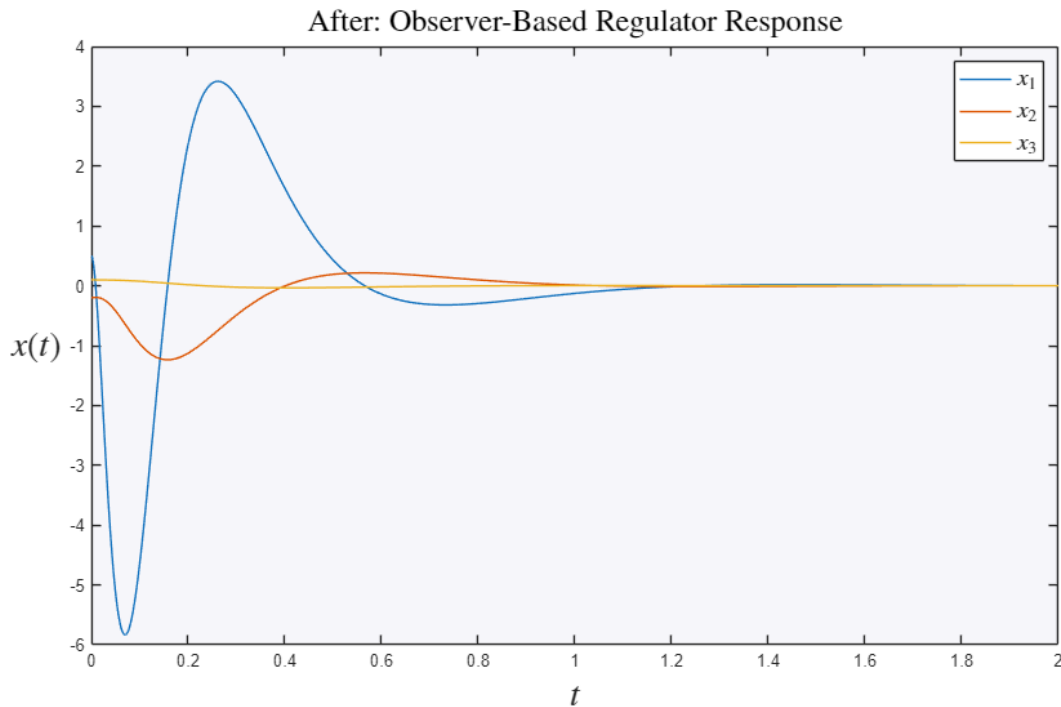
Columns 5 through 6

```
-21.0000 + 0.0000i -22.0000 + 0.0000i
```

Regulator Simulation from Nonzero Initial Condition

```
x0 = [0.5; -0.2; 0.1];
e0 = x0;
z0 = [x0;e0];
sys_aug = ss(A_aug, zeros(2*n,1), [eye(n) zeros(n)], zeros(n,1));
t = 0:0.001:2;
[x_resp,~] = initial(sys_aug,z0,t);

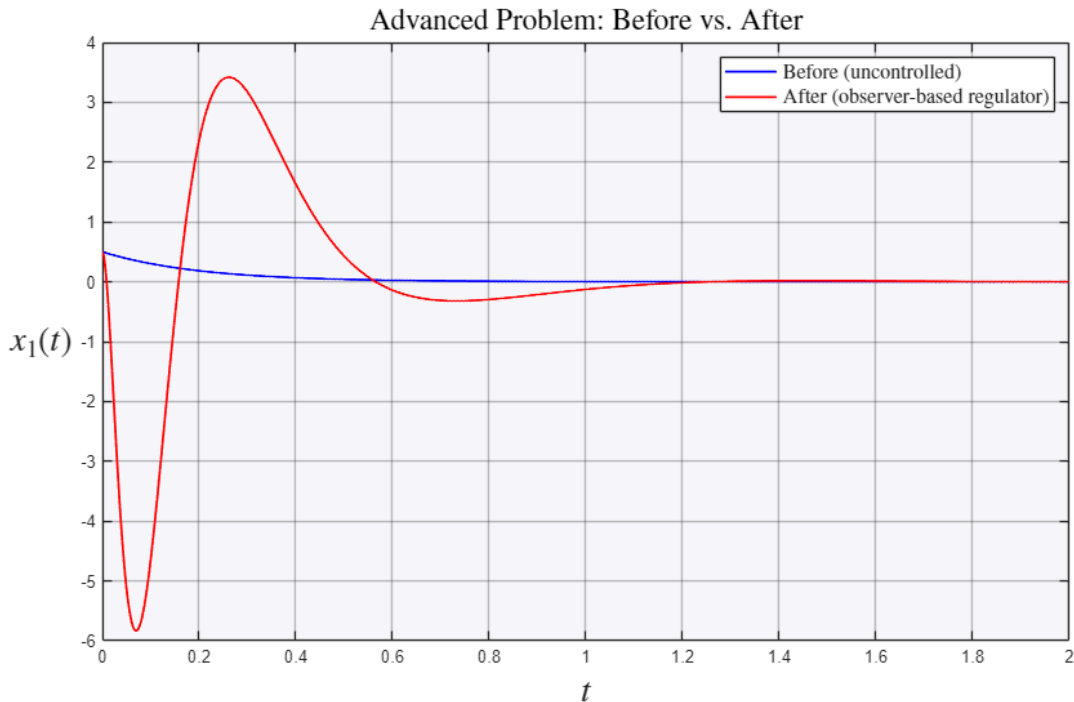
figure
plot(t,x_resp)
legend('$x_1$', '$x_2$', '$x_3$', 'Interpreter', 'latex', 'FontSize', 14)
title('After: Observer-Based Regulator Response', 'Interpreter', 'latex', 'FontSize', 18)
ylabel('$x(t)$', 'Interpreter', 'latex', 'FontSize', 20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)
```



Before vs. After: Uncontrolled Plant vs. Observer-Based Regulator

Without control the plant drifts (it has a pole at the origin); the observer-based compensator – using only the measured output y – drives every state back to zero.

```
x_ol = initial(ss(A,zeros(n,1),eye(n),0), x0, t);
figure
plot(t, x_ol(:,1),'b','LineWidth',1.3)
hold on
plot(t, x_resp(:,1),'r','LineWidth',1.3)
hold off
grid on
legend('Before (uncontrolled)', 'After (observer-based regulator)', 'Interpreter', 'latex', 'FontSize', 12)
title('Advanced Problem: Before vs. After', 'Interpreter', 'latex', 'FontSize', 17)
ylabel('$x_1(t)$', 'Interpreter', 'latex', 'FontSize', 20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)
```



Closed-Loop Transfer Function via the Compensator

Verify that closing the loop around the plant with the dynamic compensator $G_c(s)$ reproduces the same pole set as the augmented state-space system above.

```
T_closed = feedback(G*Gc,1);
fprintf('\nClosed-loop poles via feedback(G*Gc,1):\n')
disp(pole(T_closed))
```

```
Closed-loop poles via feedback(G*Gc,1):
Columns 1 through 4
-42.3536 + 0.0000i -18.1637 -23.3199i -18.1637 +23.3199i    5.3942 + 0.0000i

Columns 5 through 6
-4.1049 + 0.0000i -3.6084 + 0.0000i
```

Try it yourself

- Speed up only the observer poles and notice the regulator response barely change while the estimation transient shortens.
- Confirm `pole(feedback(G*Gc,1))` matches the controller-union-observer pole set you designed.